

Coverage-Free Fuzzing: LLM-Guided Refinement of Grammar-Based Test Generators

Anonymous Author(s)

Submission under double-anonymous review

Abstract—Property-based testing tools such as Hypothesis can turn a formal grammar into a generator of syntactically valid inputs, but the generator itself is typically static: hand-tuned once and reused unchanged for the rest of a fuzzing campaign. We study whether a large language model (LLM), given only a formal grammar and parser-level feedback – with no coverage instrumentation – can iteratively refine such a generator. We build a fully black-box pipeline targeting the cJSON parser (pinned at v1.7.19): a baseline Hypothesis strategy derived directly from an ANTLR JSON grammar generates inputs, a sanitizer-instrumented harness classifies each as accepted, rejected, or crashing, and a refinement loop feeds an LLM a summary of three observable proxy signals – acceptance rate, the structural diversity of the shapes produced, and the distinct rejection signatures seen – asking it to propose a revised generator. Every proposal is statically sandboxed (AST-checked, import-restricted) and validated before it ever touches the target binary. Across fifteen independent, seeded runs of up to five refinement iterations each, LLM-guided refinement raises mean acceptance rate from 58.2% to 97.1% relative to a static baseline, using no coverage data at all. Structural fingerprints and rejection signatures are reported as descriptive signals; because the refined arm executes more iterations, their aggregate counts are not treated as like-for-like effects. A follow-up ablation over which subset of this feedback the proposer sees – counts alone, counts with rejection signatures, or the full signal – finds acceptance-rate gains are broadly similar across the three conditions, with full feedback preserving more rejection-signature diversity than counts alone (four usable runs per condition; reported descriptively, not tested). A pre-registered replication on a second parser, parson, with the same proposer and design finds the opposite: refinement lowers mean acceptance from 95.6% to 92.4% ($p \approx 0.024$), and no crash occurs in 25,500 sanitizer-instrumented executions. Every rejected input is explained: parson accepts any bytes after a complete value, so the static baseline starts near its ceiling, while the refined generators produce grammar-valid constructs that parson refuses without reporting why. The acceptance proxy is thus only as useful as the target is strict and informative about rejections. We discuss how grammar-implementation mismatches – for instance, duplicate JSON object keys accepted by one parser and rejected by the other under an identical grammar – are themselves a source of fuzzing-relevant signal that a purely formal grammar cannot supply, and we release the full pipeline, harnesses, and reproducibility tooling.

Index Terms—fuzzing, grammar-based testing, property-based testing, large language models, black-box testing, software testing

I. Introduction

Parsers are among the most exposed pieces of code in any system: they are the first code to touch untrusted input, and a memory-safety bug in a parser is directly

attacker-controlled. Fuzzing – automatically generating inputs and observing how a target handles them – is the standard defense, but the two dominant families of fuzzer make different tradeoffs around structure and feedback. Coverage-guided, mutation-based fuzzers such as AFL discover interesting inputs by instrumenting the target and rewarding inputs that reach new code, but for a structured format such as JSON, random byte-level mutation of a seed corpus destroys syntactic validity far more often than it discovers it, so most of the fuzzer’s budget is spent on inputs the parser rejects in its first few bytes. Grammar-based and property-based approaches invert this: a generator is written directly against a grammar or schema, so every input is syntactically valid (or deliberately near-valid) by construction, and the fuzzer’s budget goes toward exploring the target’s structural handling of the format rather than re-discovering its lexer. Hypothesis, the property-based testing library we build on, is one such approach: a developer composes strategies – generators for values of a given shape – that can mirror a grammar directly.

The gap is that a hand-written grammar-based generator is usually static. It is authored once, and unless a human revisits it, it keeps producing the same distribution of shapes for the rest of a campaign, no matter what the target’s actual behavior turns out to be. Coverage-guided fuzzers close an analogous loop by using instrumentation feedback to reward and retain interesting inputs; grammar-based generators typically have nothing equivalent, because the whole appeal of generating from a grammar is that it does not require instrumenting or even recompiling the target. This is not a minor convenience: many realistic targets – vendored third-party libraries pinned at a specific commit, closed-source or license-restricted binaries, embedded or cross-compiled parsers – cannot be rebuilt with coverage instrumentation at all, or the instrumented build would not be representative of what actually ships. A feedback loop for grammar-based fuzzing is only useful in practice if it can run under exactly this constraint: black-box, with no coverage signal available.

This paper asks a narrow, empirical question under that constraint: given only a formal grammar and the kind of feedback a black-box harness can actually produce – whether an input was accepted or rejected, what the parser’s own rejection reason was, and whether a sanitizer

fired – can a large language model (LLM) act as an effective iterative refiner of a grammar-based generator, without ever seeing which lines of the target ran? We answer it by building a complete pipeline around the cJSON parser (pinned at v1.7.19) and an ANTLR-derived JSON grammar: a sanitizer-instrumented harness classifies every generated input as accepted, rejected, or crashing; a campaign layer reduces a run of the harness to three observable, coverage-free proxies – acceptance rate, the number of distinct structural shapes produced, and the number of distinct rejection reasons seen – and a refinement loop feeds those proxies, together with the grammar text, back to an LLM each iteration and asks it to propose a revised Hypothesis strategy. Because the proposer is an LLM and its output is executable code, every proposal is statically validated – AST-checked for disallowed imports and calls, and sanity-checked that it actually produces well-formed output – before it is ever run against the target binary.

We evaluate this pipeline with a seeded, fifteen-run-per-arm comparison against a static baseline generator; with a pre-registered replication of that comparison on a second, independent parser (parson), to test whether the effect generalizes beyond one target and one set of grammar adaptations; and with a smaller ablation over which subset of the proxy signal is exposed to the proposer. Concretely, this paper contributes:

- A black-box, coverage-free feedback design – acceptance rate, structural diversity, and rejection signatures – shown sufficient to steer LLM-driven refinement of a grammar-based generator without any coverage instrumentation.
- A pipeline that treats LLM-authored code as untrusted by construction: every proposed generator is statically sandboxed and behaviorally sanity-checked before it ever executes against a real target, independent of which LLM produced it.
- A seeded, reproducible, fifteen-run-per-arm empirical comparison showing that refinement raises mean acceptance rate from 58.2% to 97.1% relative to a static baseline on the same target, with complete rank separation between arms. Structural fingerprints and rejection signatures are reported as descriptive signals, with explicit caveats that unequal per-arm budgets prevent direct comparison of their aggregate counts.
- A pre-registered, fifteen-run-per-arm replication on a second parser (parson) in which refinement significantly lowers acceptance (95.6% to 92.4%, $p \approx 0.024$), with every rejected input traced to its cause: parson’s leniency toward trailing bytes leaves the baseline near its ceiling, while the refined generators produce grammar-valid constructs that parson rejects without reporting why. No crash occurred in 25,500 sanitizer-instrumented executions.
- A feedback-signal ablation isolating which subset of the proxy signal drives the improvement: exposing only acceptance/rejection counts, counts plus rejection signatures, or the full signal all raise acceptance rate by a broadly similar amount, while the full signal preserves visibly more rejection-signature diversity than counts alone – a small-sample (four runs per condition), descriptive result, not a tested effect.
- An empirical account of grammar-implementation mismatches across both targets under one shared formal grammar – for example, duplicate JSON object keys accepted by cJSON and rejected outright by parson – which we argue are themselves a source of fuzzing-relevant signal that a purely formal grammar specification cannot supply on its own.
- Reproducibility tooling for the generated-input stream specifically (a seeding shim over Hypothesis’s internal random state, per-run manifests, and hash-identified harness binaries), together with an explicit statement of what that reproducibility does and does not cover.

The rest of the paper is organized as follows. Section II situates this work relative to coverage-guided, grammar-based, and LLM-assisted fuzzing. Section III describes the pipeline, the proxy signal, and the sandboxing of LLM-proposed generators in detail. Section IV reports the cJSON comparison, the parson replication, and the feedback ablation. Section V discusses limitations, threats to validity, and what the negative result on parson does and does not show. Section VI concludes.

II. Related Work

a) Coverage-guided fuzzing.: American Fuzzy Lop [1] and its successor AFL++ [2] popularized mutation-based, coverage-guided fuzzing: a target is instrumented to report which code ran, and the fuzzer retains and mutates inputs that reach new coverage. Manès et al.’s survey [3] gives a taxonomy of this and related approaches. Coverage-guided fuzzers are highly effective when instrumentation is available, but for a structured input format their byte-level mutations mostly produce syntactically invalid inputs that are rejected before any interesting code runs, and none of this family applies when the target cannot be recompiled with instrumentation – the setting this paper addresses.

b) Grammar-based and generation-based fuzzing.: Generating inputs directly from a grammar, rather than mutating bytes, is a much older idea: Purdom’s sentence generator [4] produces short strings that exercise every grammar production, originally for testing and debugging parsers themselves. More recent tools combine a grammar with coverage feedback: Superion [5] extends a greybox fuzzer with grammar-aware mutation operators so mutated inputs remain syntactically valid, and Nautilus [6] similarly pairs a grammar-based generator with coverage-guided mutation to reach deeper parser states. Both still assume coverage instrumentation is

available to decide which grammar-derived inputs to keep; our setting removes that assumption entirely; the only feedback available is the kind a black-box harness can itself produce, so the loop steers the grammar-based generator, not a corpus of retained inputs.

c) Property-based testing.: QuickCheck [7] introduced property-based testing: a developer writes generators for values of a given shape and a property that should hold for all of them, and the tool searches for a counterexample. Hypothesis [8], which this work builds directly on, is a Python implementation of the same idea, with composable strategies that can mirror a formal grammar closely enough that our baseline generator is a near-direct translation of an ANTLR grammar. In this line of work the generator is authored once by a human and is not revised in response to how the target under test actually behaves; this paper automates exactly that revision step.

d) LLMs for fuzzing and test generation.: LLMs have recently been used to generate or mutate fuzzing inputs directly. TitanFuzz [9] uses an LLM to generate and mutate programs for fuzzing deep-learning library APIs, relying on the LLM’s prior exposure to real usage examples of those APIs rather than a formal grammar. Fuzz4All [10] generalizes this to arbitrary languages/systems by using an LLM both to synthesize a distilled input prompt from documentation and examples and to generate and mutate inputs from it. CodaMOSA [11] takes a related but distinct approach for unit-test generation: search-based test generation runs until coverage plateaus, at which point an LLM is asked for example test cases to help the search escape the plateau – i.e., the LLM is invoked because coverage feedback is available and has stalled. Closest to our loop, G2Fuzz [12] has an LLM synthesize and mutate input generators for non-textual formats and pairs them with a coverage-guided mutation fuzzer, and ELFuzz [13] evolves generation-based fuzzers with an LLM under coverage guidance. Both revise generators, as we do, but select them by coverage. Our setting differs from all of these along the same axis: we assume no coverage signal is available at any point, only the parser’s own accept/reject decision and sanitizer output, and the LLM’s job is not to produce one-off inputs or test cases but to revise a reusable, executable generator against a formal grammar it is also given directly, iterating on the same three coverage-free proxies each round.

e) Positioning.: This paper sits at the intersection of the second and fourth lines above: like Superion and Nautilus, it uses a grammar to keep generated inputs structurally valid; unlike them, it has no coverage instrumentation to fall back on, and unlike TitanFuzz, Fuzz4All, and CodaMOSA, the LLM never sees coverage data, execution traces, or usage examples, and unlike G2Fuzz and ELFuzz, no coverage signal decides which generator survives – only the grammar text and the same three black-box proxies (acceptance rate, structural diversity,

rejection signatures) a human maintaining a Hypothesis strategy by hand would have to reason about manually.

III. Methodology

A. Research questions

This paper asks three questions. RQ1: using only black-box, coverage-free feedback, does LLM-guided refinement of a grammar-based generator improve that generator’s acceptance rate and structural diversity relative to a static baseline, on a fixed target? RQ2: run unmodified except for the harness, does the pipeline carry over to a second, independently implemented parser, and does RQ1’s acceptance-rate effect replicate there? RQ3: of the three coverage-free proxies exposed to the proposer, does the specific subset shown – counts alone, counts with rejection signatures, or the full signal – change how effectively refinement raises acceptance rate?

B. Target, grammar, and harness

The primary target is cJSON [14], pinned at v1.7.19 (commit c859b25). The grammar is the ANTLR [15] grammars-v4 JSON grammar [16], vendored unmodified. A thin C harness reads a candidate input from stdin and parses it with cJSON_ParseWithLengthOpts(..., require_null_terminated=1), the library’s strictest entry point: it rejects any input with trailing bytes after a valid value, so an “accepted” input is one where the entire buffer is a single grammar-valid value, matching the grammar’s `json : value EOF ;` production exactly. The library’s more commonly used cJSON_Parse entry point does not enforce full-buffer consumption and was deliberately not used, since it would silently accept inputs the grammar does not.

Reconciling the formal grammar with the target’s actual behavior surfaced three adaptations, each a genuine mismatch between what the grammar specifies and what the implementation does: (i) the grammar has no token for NaN/Infinity, matching cJSON’s inability to produce or accept them, so generators exclude non-finite floats; (ii) cJSON accepts duplicate object keys, storing every entry in an internal linked list rather than rejecting or de-duplicating, even though the grammar is silent on duplicate keys – this is treated as valid-but-underspecified territory, not a rejection case, and generators deliberately exercise it; (iii) the full-buffer-consumption requirement above is enforced by the harness’s choice of entry point, not by the grammar itself. Section IV reports the analogous set of adaptations found for the second target, parson, where two of the three go the opposite direction under the identical grammar.

C. Outcome classification

Every input is executed as a bounded subprocess (5-second default timeout) against a sanitizer-instrumented build (-fsanitize=address,undefined). The result is classified, in order: timeout if the process does not return within

the bound; signal:<name> if it is killed by a fatal signal; crash if it exits with a non-zero return code or its stderr contains AddressSanitizer or UndefinedBehaviorSanitizer text; accepted if its stdout begins with status=accepted; otherwise rejected. Timeouts are classified and reported alongside crashes rather than discarded, since an unbounded hang in a parser is a denial-of-service bug in its own right.

D. A coverage-free proxy signal

With no coverage instrumentation, refinement is steered by three signals computed purely from a campaign’s classified outcomes:

- Acceptance rate – the fraction of inputs classified accepted. A generator accepting close to 0% of its own output is testing only the rejection path, not the parser’s structural handling of valid input, and is flagged for correction.
- Structural diversity – the count of distinct structural fingerprints seen in a campaign. A fingerprint maps each byte of an input to one of six symbols (the six structural characters `{ } [ ] : ,` are kept literal, whitespace maps to `_`, ASCII digits map to `#`, the double-quote maps to `"`, and every other byte maps to `x`), concatenates the result, and truncates it to 256 characters. Two inputs share a fingerprint exactly when their first 256 bytes reduce to the same positional shape string; the count of distinct fingerprints across a campaign is used as a coverage-free proxy for how many different grammar shapes a generator has actually produced. The 256-byte truncation makes this proxy a lower bound on true shape diversity for inputs longer than that, since two structurally different inputs that agree on their first 256 bytes collapse to one fingerprint; this is a deliberate accuracy/cost tradeoff and is stated as a limitation in Section V.
- Rejection signatures – the count of distinct parser rejection messages and offsets seen, used to tell the proposer which malformed shapes are already well covered versus unexplored.

These three numbers, plus outcome counts and unique input lengths, are the entire feedback surface exposed to the LLM each iteration; no coverage data, execution trace, or source excerpt is ever included in the prompt.

E. Refinement loop

Each iteration builds a prompt containing the grammar text and a summary of the previous iteration’s campaign (outcome counts, unique input lengths, unique structural fingerprints, unique rejection signatures), and sends it to the LLM proposer. The proposer must return a single Python function, `generated_json`, decorated with Hypothesis’s `@st.composite` and built from `st.*` combinators, so that recursive structure is expressed the same way a human-authored Hypothesis strategy would express it.

Every reported run uses gpt-4.1-mini at temperature 0.2 as the proposer.

F. Sandboxing untrusted, LLM-authored code

Because the proposer’s output is executable Python that this pipeline runs locally, every proposal passes through a validator (`load_strategy`) before it ever touches the target binary. The validator first parses the proposal’s AST and rejects it outright if it imports anything other than `hypothesis.strategies`, or if it calls `eval`, `exec`, `open`, `compile`, or `__import__` by name. The proposal is then executed with an otherwise ordinary Python builtin namespace, minus a short, explicit blocklist (`eval`, `exec`, `compile`, `__import__`, `open`, `input`, `breakpoint`, `exit`, `quit`, `help`, `globals`, `locals`, `vars`, `getattr`, `setattr`, `delattr`) – ordinary helpers such as `ord`, `chr`, `str`, `len`, and `range` remain available, so a proposal is free to use normal Python rather than being restricted to Hypothesis combinators alone. Finally, the resulting `generated_json` must be callable and must produce a bytes value via `.example()`; a proposal that fails any of these checks is rejected and logged, and never reaches the harness. This is deliberately scoped as a filter against accidental misuse of untrusted, LLM-authored code, not as a hardened security boundary: Python’s object model can reach functionality beyond any blocklisted name (for instance, by walking `().__class__.__base__.__subclasses__()`), and closing that off would require process- or container-level isolation rather than name filtering. We report this limitation directly rather than implying a guarantee the design does not provide.

G. Crash triage and minimization

A sanitizer report is reduced to its SUMMARY: line and stack frames with addresses and source line numbers normalized out, then hashed into a 16-character signature; records sharing a signature are treated as one root cause. For each unique signature, Hypothesis’s `find` is used to shrink a triggering input to a minimal byte string that still reproduces the same crash or timeout classification – rather than simply keeping the first crash observed – and the minimized input is re-run once, standalone, against the pinned build to verify it reproduces outside the shrinking process itself.

H. Reproducibility engineering

Hypothesis’s `.example()` draws from a private internal random state that Python’s own `random.seed()` does not reach, and separately orders its candidate batch with the module-level `random.shuffle`. Seeding only one of the two leaves the produced example stream non-reproducible across process invocations; a shim (`seeding.seed_everything`) seeds both, which we verified gives byte-identical example streams across repeated processes on Hypothesis 6.165.5 / CPython 3.14.7. Two documented, non-internal alternatives – `settings(derandomize=True)` and `global_force_seed` – were

rejected because both collapse a run’s draws into repeats of a single small batch, which would change the generation algorithm itself rather than only fix its starting point. Because this shim depends on an internal implementation detail, the Hypothesis version is pinned in requirements.txt and the shim fails loudly rather than silently if the relied-upon attribute is renamed or removed in a future release. Every run additionally persists a manifest (seed, package versions, platform, model, feedback mode, CLI arguments) and a reproducibility report (compiler, harness SHA-256, exact command line, repository commit); the compiled harness binary itself is gitignored and host-compiled, so it is identified by content hash rather than assumed identical across machines. This reproducibility guarantee is scoped to the generated input stream only: it does not extend to LLM proposals themselves (temperature 0.2, no server-side seed), to wall-clock-dependent timeout classification, or to the harness binary’s bit-for-bit identity across hosts and toolchains.

I. Experimental design

For RQ1, we run fifteen independent, seeded trials (seeds 0–14) of a static baseline generator and fifteen independent, seeded trials of the LLM-refined loop against the same pinned cJSON build, each trial consisting of up to five refinement iterations of up to 500 examples. We report acceptance rate, structural fingerprints, and rejection signatures per trial. We compare the arms statistically only on acceptance rate, the scale-free outcome; the count-based metrics are retained as descriptive feedback signals because the two arms execute different total numbers of inputs. For RQ2, we repeat RQ1’s design on parson exactly – fifteen seeded runs per arm, seeds 0–14, the same baseline, proposer, budget, and feedback – with only the harness changed. The protocol, the primary test (exact two-sided Mann–Whitney U on per-run acceptance rate), and the reporting rules were committed to the artifact before any run. An earlier exploratory pilot on parson, whose proposals were written outside the loop, is reported separately (Section IV-C). For RQ3, we repeat the RQ1 refined-arm recipe under each of the three feedback modes in Section III-D (counts, counts+rejections, full), five seeded trials (seeds 0–4) per mode against the same cJSON build. In one trial per mode, every one of the five iterations produced an LLM proposal that failed sandbox validation, leaving that trial with no executed inputs; following the same `iterations_with_data` convention used throughout, these three trials are excluded, leaving four usable trials per mode.

IV. Evaluation

A. RQ1: cJSON, baseline versus LLM-refined

Table I reports the fifteen-seeded-run-per-arm comparison described in Section III – an extension of an initial five-run-per-arm pilot to a larger sample, run identically. Every refined-arm run has a higher acceptance rate than

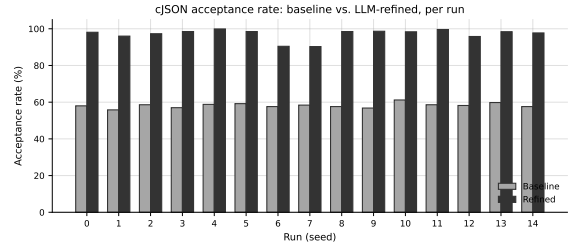


Fig. 1. Per-run acceptance rate, baseline vs. refined, cJSON (fifteen seeded runs per arm). Every refined run exceeds every baseline run.

every baseline-arm run. Since acceptance rate is scale-free, this arm-level comparison remains interpretable despite the different numbers of executed inputs. All fifteen observations have complete rank separation, giving an exact two-sided permutation p-value of $2/\binom{30}{15} \approx 1.29 \times 10^{-8}$ for this run-level comparison. This is the smallest attainable exact p-value for a design with $n=15$ per arm, and, as with the smaller pilot, should be read as the strongest separation this design can report rather than as a precisely estimated population effect. Structural fingerprints and rejection signatures remain useful feedback signals, but their aggregate counts are not tested as arm-level effects because the refined runs execute more iterations.

Unequal budgets. Baseline runs execute 500 inputs on average; refined runs execute 1,533 on average, because each refined run has up to five iterations of 500 inputs and an iteration whose LLM proposal is rejected executes none (29 of the 75 refined iterations). Every count-based quantity (accepted, rejected, fingerprints, signatures as raw totals rather than rates) therefore scales with a budget that differs between arms and is not directly comparable; acceptance rate is scale-free and unaffected by this. We report the budget-sensitive metrics above as sums over a run’s iterations specifically so the comparison is transparent about this rather than implying parity.

No crashes in either arm. Zero sanitizer crashes, timeouts, or fatal signals occurred in either arm across all thirty runs. The effect measured here is therefore on generator quality – validity and structural diversity – not on vulnerability discovery in cJSON; we do not claim or imply that refinement found, or was likely to find, a memory-safety bug in this target.

B. RQ2: replication on parson

We reused every pipeline component unchanged against a second, independent target, parson [17] (commit ba29f4e, v1.5.3); only the harness differs. Reconciling the same JSON grammar against parson’s real behavior surfaced three adaptations, two of which go the opposite direction from the cJSON adaptations reported in Section III under the identical grammar: (i) unlike the cJSON harness, parson’s parse entry point does not require full-buffer consumption, so trailing garbage after a valid value is silently accepted – a superset of what the

TABLE I

cJSON: baseline vs. LLM-refined, fifteen seeded runs per arm (mean over runs). The exact two-sided permutation $p \approx 1.29 \times 10^{-8}$ applies only to the scale-free acceptance-rate comparison; fingerprint and rejection signature totals are descriptive because budgets differ.

Metric	Baseline	Refined	Change
Acceptance rate	58.21%	97.07%	+38.86 pp
Structural fingerprints (sum/run; descriptive)	274.5	598.2	—
Rejection signatures (sum/run; descriptive)	87.8	24.1	—
Crashes / timeouts / signals	0	0	—

TABLE II

parson: baseline vs. LLM-refined, fifteen seeded runs per arm, identical design to Table I. p : exact two-sided Mann–Whitney U on per-run acceptance rate.

Metric	Baseline	Refined
Acceptance rate, mean (sd)	95.61% (0.95)	92.41% (3.96)
Acceptance rate, per-run range	94.0–97.0%	86.7–98.7%
Crashes / timeouts / signals	0	0
Distinct rejection signatures	1	1
Refined – baseline: -3.20 pp, $p \approx 0.024$, $\delta = -0.48$		

grammar’s json : value EOF ; production specifies; (ii) parson explicitly rejects any object containing a duplicate key, propagating the rejection up as a failure for the whole object – a subset of what cJSON accepts, even though the grammar itself is silent on duplicate keys for both; (iii) parson enforces a fixed nesting limit (MAX_NESTING = 2048), rejecting deeper structures outright rather than exhibiting unbounded recursion. That the same formal grammar under-determines a directly opposite validity decision (duplicate keys: accept vs. reject) on two independently implemented parsers is, on its own, evidence that a purely formal grammar specification cannot supply the target-specific signal needed to fuzz either implementation precisely.

Replication. Table II reports the pre-registered replication of RQ1’s design on parson. Refinement lowered acceptance: 92.4% against the baseline’s 95.6% (exact two-sided Mann–Whitney $p \approx 0.024$, Cliff’s $\delta = -0.48$). The baseline has tied values, which the exact test assumes away; a tie-corrected test ($p \approx 0.025$) and a permutation test on the difference in means ($p \approx 0.005$) agree. No crash, timeout, or signal occurred in 25,500 sanitizer-instrumented executions, and 39 of the 75 refined iterations lost their proposal to sandbox validation (29 of 75 on cJSON).

Why refinement lowers acceptance on parson. Every input parson rejected in either arm is explained by a construct confirmed with a minimal document against the harness. The baseline alternates a grammar-valid document with the same document plus one trailing suffix (a comma, a bracket, a brace, a word, or a NUL byte). cJSON’s full-buffer entry point rejects every suffix except NUL, which is what gives it room to improve; parson accepts any bytes after a complete value, so none is rejected,

and its only baseline rejections (329 of 7,500) are objects whose key contains an escaped NUL, which parson’s source states it does not support. The baseline therefore starts at 95.6% with almost nothing for refinement to fix. The refined generators are more varied and produce grammar-valid constructs that parson refuses. Of the 1,063 inputs it rejected, 474 contain a duplicate key, 427 an unpaired surrogate escape, 138 a number that overflows a double, 136 a zero followed directly by an exponent (0e5), and 10 a NUL in a key (an input can have several); only 6 are not valid under the grammar. The last two numeric cases differ in status: RFC 8259 permits limits on number range, but 0e5 is valid JSON, and support for it was proposed to parson and not merged [18]. A further 392 inputs (2.2%) failed to encode and count as not accepted, as in RQ1. Finally, parson’s parser returns no error detail, so all 1,392 rejections in both arms share one rejection signature: the proxy tells the proposer how many inputs failed, never why.

C. Exploratory parson pilot

Before API access was available, an exploratory pilot ran five iterations on parson whose proposals were not produced by the loop’s LLM proposer: they were written outside the loop, in an interactive session with an AI coding assistant that could read parson’s source code. The pilot is therefore neither LLM-proposer-driven nor black-box, and we report it only for what it targeted and found, not as evidence about the method. Table III summarizes its iterations.

No crash was found in the pilot’s 3,000 sanitizer-instrumented executions (500 baseline plus 5×500 pilot iterations), verified by grepping every run’s stderr for AddressSanitizer, UndefinedBehaviorSanitizer, and LeakSanitizer text and checking for any non-accepted/rejected status. A source review of the highest-risk code paths – parse_utf16’s raw pointer walk across surrogate pairs, process_string’s output-buffer sizing, json_object_grow_and_rehash’s linear probing, and, informed specifically by iterations 4–5, json_object_add’s duplicate-key failure branch and parse_object_value/parse_array_value’s missing-bracket failure branch – found that each relies on the harness’s own null terminator as a safe, in-bounds stopping condition, and that internal cleanup calls to json_value_free are consistent with the harness’s own top-level call (an object’s

TABLE III

parson, exploratory pilot (not part of the statistics): five iterations of 500 examples each, with proposals written outside the loop.

Iter.	Focus (abridged)	Acc./Rej.	Fingerprints
1	Grammar-seeded generator: recursion, numeric edge cases, duplicate keys, trailing garbage	285 / 215	206/500
2	Wide objects/arrays (up to 4000 keys) to hit rehash/resize paths unreached by iter. 1	341 / 159	273/500
3	Hash-bucket collisions, combined wide+deep structures, surrogate escapes at closing quote	442 / 58	254/500
4	Internal error-cleanup free path: structures valid until the final token	85 / 415	214/500
5	Combines iter. 4’s free-path shapes with a sibling-cascade free case	139 / 361	302/500

key count is never incremented before a duplicate is detected, so cleanup never iterates past the last successfully added slot). We report this as a defensible, evidence-supported explanation for the absence of a crash within this budget, not as proof that none exists: five iterations against one pinned commit, exercising the parse and internal free paths but not parson’s public mutation API (`json_object_remove`, `json_array_remove`, and the `_replace_*` family), is a bounded search, and we do not claim more than that bound supports.

D. RQ3: which feedback signal drives the improvement

Table IV reports the feedback-signal ablation described in Section III: the same refined-arm recipe run under each of the three feedback modes, five seeded trials (seeds 0–4) per mode. In one trial per mode, all five refinement iterations produced an LLM proposal that failed sandbox validation (Section III), leaving that trial with no executed inputs; these three trials are excluded, leaving four usable trials per mode – the same proposal-rejection behavior visible throughout RQ1’s refined arm, just concentrated into a full trial in these smaller-sample runs rather than spread across otherwise-successful trials.

All three modes raise acceptance rate to a broadly similar, near-ceiling level – exposing only acceptance/rejection counts is already close to sufficient for that specific objective, and the per-trial acceptance rates across modes overlap (Table IV), so we do not claim any mode is statistically distinguishable from another on this metric at $n=4$. The one pattern worth reporting descriptively is rejection-signature diversity, which increases monotonically with how much of the signal the proposer sees: counts alone, with no visibility into which rejection shapes are already covered, produces under a fifth of the distinct rejection signatures that full feedback does, consistent with – though not proof of – the concern raised in Section V that a proposer optimizing acceptance rate without visibility into rejection diversity has less reason to keep producing malformed, boundary-probing input. At four trials per condition, this is a suggestive, small-sample result, not a tested effect, and we report it as exactly that.

V. Discussion

A. Interpreting the effect on cJSON

The result in Section IV says something narrower than “the LLM makes fuzzing better”: across fifteen seeded

runs, refinement reliably moved a generator from producing mostly-invalid input toward producing mostly-valid input, using only the three coverage-free proxies it was given. That acceptance-rate effect is real and statistically clean at the sample size tested – the scale-free metric on which the two arms are directly comparable – but it is worth being explicit about what it does not show. A 97.1% mean acceptance rate is close to a ceiling, and a generator that always emits valid input has, by construction, stopped exercising the accept/reject boundary at all – exactly the region the project’s own `near_valid_json` baseline strategy exists to probe. Acceptance rate rising is desirable when the baseline is far from valid and mostly testing the rejection path, which is the regime our baseline started in; it is not obviously desirable in the limit, where a generator that never produces a rejected or malformed input has traded away exactly the boundary-probing behavior that historically finds parser bugs. We did not observe this failure mode directly – within the refined arm, structural fingerprint counts also rose and rejection signatures did not collapse to a single trivial case, though these counts are descriptive rather than tested against the baseline (Section IV) – but the proxy signal as designed rewards acceptance rate monotonically and has no explicit floor protecting the near-valid regime. A refinement objective that also rewarded a target reject rate, rather than treating “more accepted” as strictly better without limit, would be a direct, low-cost improvement to the design evaluated here.

B. Interpreting the parson result

The replication shows that the acceptance proxy depends on the target in two ways. First, it needs headroom: a baseline built from trailing-byte mutations is almost fully accepted by a parser that ignores trailing bytes, so there is little left for refinement to raise. Second, it needs the target to be strict in the places the grammar is, and to say why it rejects: parson refuses grammar-valid constructs that cJSON accepts and reports no error detail, so a proposer shown only counts and a single rejection signature cannot tell that its duplicate keys or 0e5 numbers are the problem. Refinement then turns the concern raised above for cJSON around: its more varied generators reach more of the grammar, and on this target that costs acceptance. For fuzzing this is not wasted effort, since those inputs exercise parson’s rejection paths, but it

TABLE IV

cJSON refined arm under three feedback modes, four usable seeded trials per mode (one trial per mode excluded; see Section IV-D). Descriptive only: with $n=4$ per condition and no separation between conditions, no significance test is reported or implied.

Feedback mode	Acceptance rate	Fingerprints (sum/run)	Rej. signatures (sum/run)
counts	92.58%	505.5	6.5
counts+rejections	96.42%	457.3	26.8
full	95.53%	503.5	34.8

means acceptance rate cannot serve as the objective on its own, and it motivates a proxy that separates grammar-valid rejections from malformed input.

Finding no crash in 25,500 sanitizer-instrumented executions on parson, and none in the exploratory pilot, is evidence about this pipeline’s search within this budget on this pinned commit, not evidence about parson’s correctness in general. It cannot distinguish “no bug in the code paths we exercised” from “a bug exists in a code path we did not exercise,” such as parson’s public mutation API, which the black-box parse-then-free harness used here cannot reach at all.

What the parson runs do establish is narrower and more mechanical: every non-target-specific pipeline component (runner, campaign, proposal, refinement, triage, minimize) worked unmodified against a second parser with a different, and in several respects opposite, set of grammar adaptations, with only a new harness and build script required. The pipeline is portable across targets; the effect of its acceptance proxy is not.

C. Threats to validity

Construct validity. A “crash” in this pipeline is defined purely by sanitizer output and process signals; it cannot detect a parser that silently misparses, accepts input the grammar disallows, or otherwise violates the specification without a memory-safety fault. The grammar mismatches documented in Sections III and IV (duplicate keys, full-buffer consumption) were found by deliberately generating inputs in that region and reading the result, not by any automated oracle; a generator that never targets an under-specified region of the grammar would not surface this class of finding at all.

Internal validity. The parson replication ran 24 days after the cJSON runs, with the harness rebuilt by Homebrew clang 23.1 because a toolchain upgrade had removed the clang 22 sanitizer runtime it was linked to; acceptance depends on parson’s parsing logic, not on the sanitizer runtime, and every binary is identified by hash in its run’s reproducibility report. The structural-fingerprint proxy truncates at 256 bytes (Section III), so it systematically undercounts diversity among longer, deeply nested inputs – exactly the inputs a refinement loop chasing “more diversity” is most likely to produce over successive iterations, which could understate the true diversity signal reflected in the descriptive fingerprint counts we report.

Statistical validity. $n=15$ runs per arm is still modest; the exact test used is appropriate for that size and makes no distributional assumption, but $p \approx 1.29 \times 10^{-8}$ on the acceptance-rate comparison is a ceiling on how small an exact p -value fifteen-per-arm observations can produce, not a measure of effect size on its own, and we report the actual acceptance-rate numbers, alongside the descriptive fingerprint and rejection-signature counts, in Table I for that reason. Baseline and refined arms also executed different total numbers of inputs (Section IV), which is why only the scale-free acceptance rate is tested statistically; any reader extending this comparison with new count-based metrics should convert to a rate first.

External validity. All results are on JSON, on two relatively small and independently well-tested C parsers, with one LLM (gpt-4.1-mini) at one temperature setting for every LLM-driven arm. The two parsers already disagree on the direction of the effect; we make no claim that either magnitude, or the absence of a crash, generalizes to other formats, other parsers, or other proposer models.

D. When this approach is, and is not, the right tool

The motivating case for a coverage-free feedback loop is a target that genuinely cannot be instrumented – a vendored dependency pinned at a specific commit for compatibility reasons, a license-restricted or closed-source binary, or a cross-compiled/embedded parser where an instrumented build would not be representative of what ships. Where coverage instrumentation is available, a coverage-guided or grammar-aware coverage-guided fuzzer (Section II) has strictly more signal to work with and should be preferred; this pipeline is not proposed as a replacement for that setting, only as an option for the black-box one.

E. Future work

Several extensions are direct given the existing infrastructure. First, the feedback-signal ablation in Section IV-D was run at a small scale (four usable trials per mode) and its rejection-signature finding is reported descriptively for that reason; a larger-sample replication is the natural next step to establish whether the pattern we observed holds statistically. Second, adding an explicit reject-rate floor to the refinement prompt, motivated directly by this section’s concern about an acceptance-rate ceiling and now further motivated by the ablation’s rejection-signature pattern, could test whether

it preserves boundary-probing behavior without sacrificing the acceptance-rate gain measured here. Third, and most directly relevant to parson, driving `json_value_free` through parson’s public mutation API rather than only the parse-then-free path would exercise object lifetime code this black-box harness cannot currently reach at all. Fourth, comparing against an existing external fuzzer under an equalized time or execution budget, rather than only against this project’s own static baseline, is the most direct way to situate the magnitude of the cJSON result against prior work.

Two further extensions are larger in scope and deliberately left for future work rather than attempted here. One is broadening beyond a second same-language, same-format target: a third C JSON parser would add little to what the parson replication already shows about target dependence, whereas a target in a different format, or a memory-safe language where a “crash” oracle must be redefined around uncaught exceptions and API contract violations rather than sanitizer output – for instance, JSON deserialization in a Dart/Flutter mobile application – would test generalization along a materially different axis than implementation alone. The other is applying this pipeline to a live, running mobile or server application rather than a bounded harness process, which would require redesigning the outcome-classification step in Section III around that environment’s own failure signals.

VI. Conclusion

We asked whether an LLM, given only a formal grammar and the feedback a black-box harness can actually produce – acceptance rate, structural diversity, and rejection signatures, with no coverage instrumentation at all – can iteratively refine a grammar-based fuzzing generator. Across fifteen seeded runs against a pinned cJSON build, it reliably can: mean acceptance rate rose from 58.2% to 97.1% relative to a static baseline, with complete rank separation between arms in this scale-free outcome. Structural fingerprints and rejection signatures were useful within-campaign feedback signals, but their aggregate counts are not treated as cross-arm effects because the arms used different budgets. A feedback-signal ablation found that acceptance-rate gains hold up under a sparser signal than the full one, but that the full signal preserves more rejection-signature diversity – a small-sample, descriptive finding that motivates, without yet proving, an explicit reject-rate objective as future work. A pre-registered replication on a second parser, parson, reversed the effect: refinement lowered acceptance from 95.6% to 92.4%, because parson accepts trailing bytes, rejects several grammar-valid constructs that the refined generators produce, and reports no error detail. The pipeline ran unmodified on both targets; the value of its proxy signal depends on the target. No crash was found on either parser.

The broader point this project argues for is methodological rather than a single number: a coverage-free proxy signal, cheap enough to compute from a black-box harness’s own accept/reject decision, is sufficient to steer LLM-guided refinement of a grammar-based generator on a target that is strict and reports why it rejects input, which matters specifically for the targets coverage-guided fuzzing cannot reach – pinned vendored dependencies, closed or license-restricted binaries, and embedded or cross-compiled parsers. We also found that reconciling a formal grammar against two real, independently implemented parsers is itself informative: the same grammar under-specifies a directly opposite validity decision (duplicate object keys accepted by one implementation, rejected by the other) on two targets, which is exactly the kind of target-specific knowledge a purely formal specification cannot supply and that fuzzing against the real implementation surfaces for free.

We release the full pipeline, both harnesses, every campaign artifact, and the reproducibility tooling described in Section III, so that the comparisons in Section IV on both parsers can be checked, extended, or falsified directly rather than taken on faith.¹

Acknowledgment

Use of generative AI. An AI coding assistant, Claude Code [19], was used throughout this work: to implement the harnesses and pipeline, to run and analyze the experiments, and to draft and revise the text of every section. The OpenAI model named in Section III is the system under study, not a writing aid. The author directed the work, reviewed all content, and takes full responsibility for it.

References

- [1] M. Zalewski, “American fuzzy lop,” <https://lcamtuf.coredump.cx/afl/>, 2014, accessed 2026.
- [2] A. Fioraldi, D. C. Maier, H. Eißfeldt, and M. Heuse, “AFL++: Combining incremental steps of fuzzing research,” in 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association, 2020.
- [3] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, “The art, science, and engineering of fuzzing: A survey,” *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.
- [4] P. Purdom, “A sentence generator for testing parsers,” *BIT Numerical Mathematics*, vol. 12, no. 3, pp. 366–375, 1972.
- [5] J. Wang, B. Chen, L. Wei, and Y. Liu, “Superion: Grammar-aware greybox fuzzing,” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 724–735.
- [6] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, “NAUTILUS: Fishing for deep bugs with grammars,” in *Proceedings of the 26th Network and Distributed System Security Symposium (NDSS)*, 2019.
- [7] K. Claessen and J. Hughes, “QuickCheck: A lightweight tool for random testing of Haskell programs,” in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2000, pp. 268–279.

¹<https://anonymous.4open.science/r/agentive-grammar-fuzzing-6177/>

- [8] D. R. MacIver, Z. Hatfield-Dodds, and Many other contributors, “Hypothesis: A new approach to property-based testing,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.
- [9] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, “Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2023, pp. 423–435.
- [10] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, “Fuzz4All: Universal fuzzing with large language models,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.
- [11] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, “CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models,” in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023.
- [12] K. Zhang, Z. Li, D. Wu, S. Wang, and X. Xia, “Low-cost and comprehensive non-textual input fuzzing with LLM-synthesized input generators,” in *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [13] C. Chen, B. Dolan-Gavitt, and Z. Lin, “ELFuzz: Efficient input generation via LLM-driven synthesis over fuzzer space,” in *Proceedings of the 34th USENIX Security Symposium*, 2025.
- [14] D. Gamble, “cJSON: Ultralightweight JSON parser in ANSI C,” <https://github.com/DaveGamble/cJSON>, 2024, version v1.7.19, commit c859b25.
- [15] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.
- [16] “ANTLR grammars-v4: JSON grammar,” <https://github.com/antlr/grammars-v4/blob/master/json/JSON.g4>, 2024.
- [17] K. Gabis, “parson: Lightweight JSON library written in C,” <https://github.com/kgabis/parson>, 2024, version 1.5.3, commit ba29f4e.
- [18] “Add compatibility for scientific notation,” parson pull request #188, <https://github.com/kgabis/parson/pull/188>, 2022.
- [19] Anthropic, “Claude code,” <https://claude.com/product/claude-code>, 2026.